

Stock watchlist and price alerts

Assignment 1 report | Issam Arida | 4 October 2026

Repository: <https://github.com/issamarida/devops-a1-issamarida>

1. Problem, users and scope

The app is for retail investors who follow a handful of US stocks next to a job or their studies. They can't watch a chart all day. They want a private list of the stocks they follow with a note on why, and they want to know when a price crosses a level they care about.

I sized it for a small pilot: up to 50 accounts on one instance, each with up to 30 tracked assets and 100 alert rules. Those numbers are the configurable defaults in `app/config.py`. Each investor has a private account so nobody can read or change another person's notes or rules.

There are two business domains. **Watchlist** owns the tracked assets and their notes. **Alerts** owns one-shot threshold rules ("AAPL above 200") and an immutable history of every rule that fired. Both persist to SQLite. Authentication and the market data adapters are supporting infrastructure around the two domains and are not counted as domains.

2. Development process and SMART goals

I followed an iterative and incremental lifecycle. Each increment was a vertical slice that ran end to end before the next one started: first the scaffold and the deployment contract, then the watchlist, then alerts, then the price adapters with the background poller, then tests and documentation. This suited one developer with one week. A waterfall plan would have frozen the domain boundary on day one, and I revised that boundary twice. Scrum assumes a team with sprint ceremonies, which a solo project can't use.

Goal (specific and measurable)	Deadline	Outcome
Both domains read and write SQLite end to end	1 Oct	Met on 30 Sep
At least 70% coverage of core logic with pytest-cov	1 Oct	Met: 100% on the first measured run
<code>python app.py</code> on an empty <code>DATA_DIR</code> is healthy within 5 seconds	2 Oct	Met and enforced by a contract test
Five ADRs across at least three commit dates	2 Oct	Met: 28 Sep, 30 Sep and 1 Oct

What went less well was pacing. The repository was created on 23 September but real work started on 28 September, so ten commits landed on 30 September alone, and a few early commit messages ("update", "init") don't meet the brief's standard. The model did what it is meant to do when requirements changed late. On 3 October a review against my own target users showed that one shared anonymous watchlist was wrong for real investors, so I added private accounts and fixed a race condition in rule firing. I also replaced simulated prices with live Finnhub data. On 4 October the watchlist became a real-time dashboard. Each change was recorded as a dated revision inside the existing ADRs, because the brief asks for exactly five entries.

The history has 33 commits across 8 calendar days, and GitHub shows pushes on 7 different days. The busiest day holds 30% of all commits, under the 40% limit. Larger changes went through `feature/*` branches into `develop` and then `main`.

3. Architecture

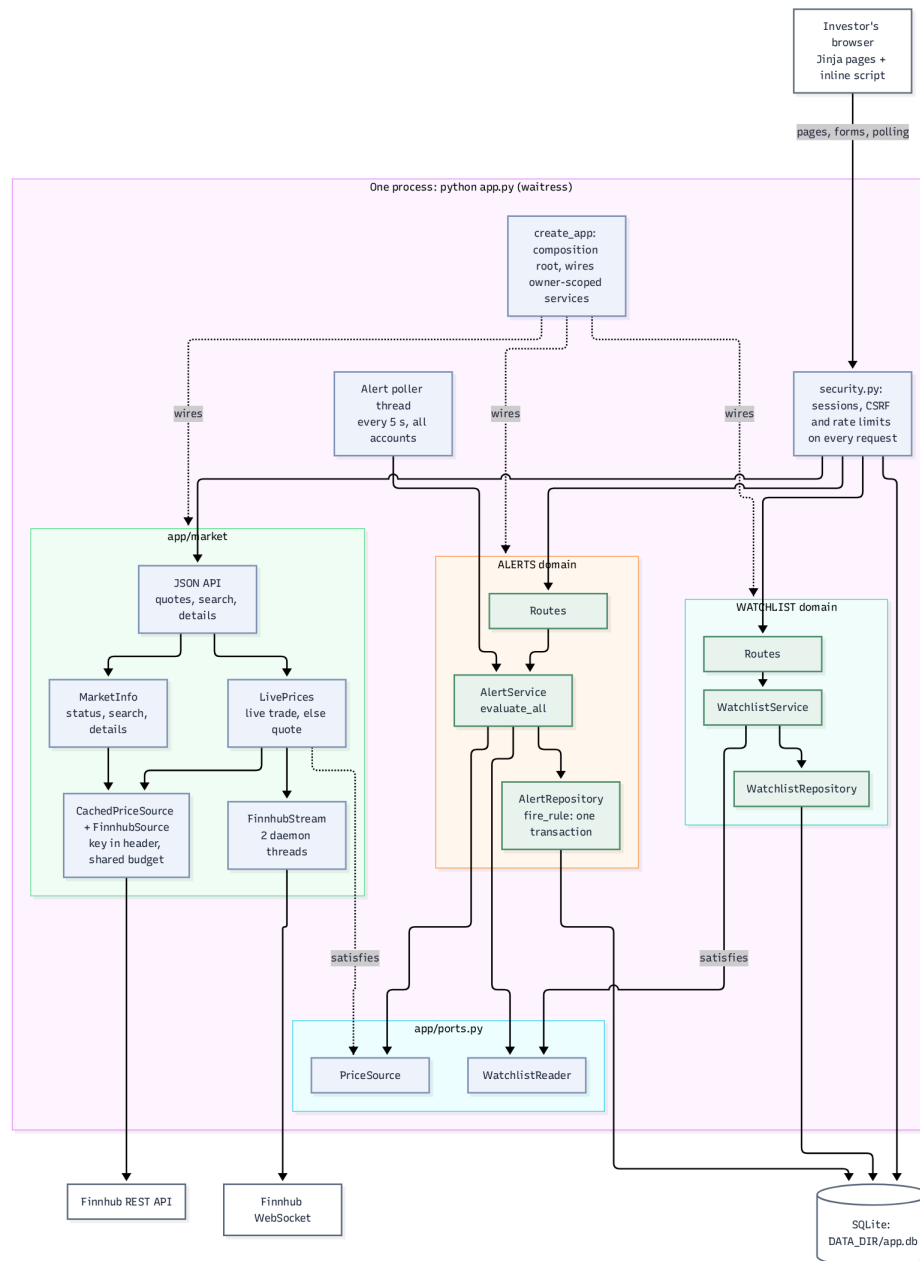


Figure 1. Component view. Green boxes are the two business domains. Solid arrows are runtime calls, dotted arrows are dependency wiring and interface conformance.

Layering. Each domain is split into routes (HTTP parsing), a service (validation and business rules) and a repository (parameterised SQL). A repository opens one short-lived SQLite connection per call, so the request threads and the background threads never share a connection.

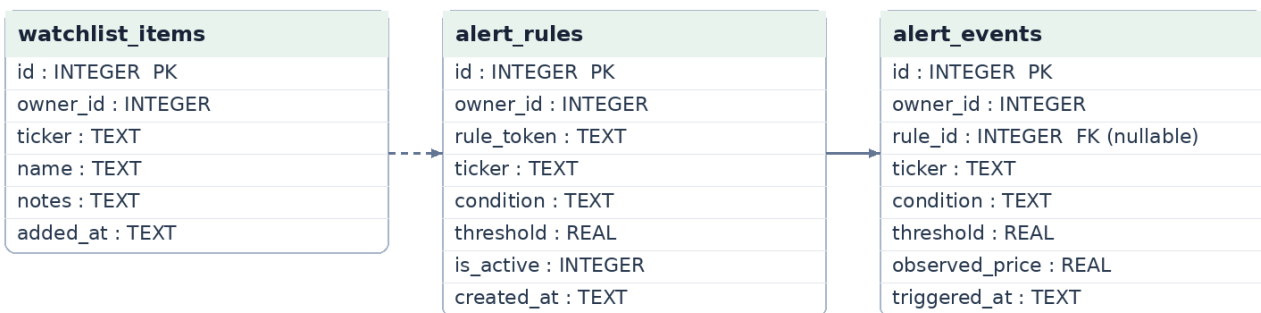
The seam (ports and adapters). Alerts needs two facts from outside its boundary: whether a ticker is on the investor's watchlist, and its current price. I expressed those as two interfaces in `app/ports.py`, `WatchlistReader` and `PriceSource`. Alerts depends only on these abstractions, which is dependency inversion. `WatchlistService` and `LivePrices` satisfy them structurally without inheriting from them. `create_app` is the single composition root that binds concrete classes to the interfaces, scoped to the signed-in account. An architecture test parses every import under `app/` with Python's `ast` module and fails the build if a domain imports the other domain or the market adapters. To extract alerts as a service in Assignment 2, `WatchlistReader` becomes an HTTP client and alerts takes its two tables and the poller with it. The service code stays as it is. Identity propagation and network failure handling would be the new work.

Concurrency model. Everything runs in one process. `app.py` starts waitress plus three daemon threads: the alert poller, which evaluates every account's active rules every 5 seconds, and two threads that maintain one WebSocket subscription to Finnhub. `create_app` never starts a thread, so tests construct the whole application without side effects.

Real-time data path. The server keeps the latest trade per subscribed ticker in memory. `LivePrices` returns a trade less than 60 seconds old, and otherwise falls back to a REST quote behind a TTL cache. Every REST call goes through one rolling budget of 50 calls per minute, below Finnhub's free limit of 60. Ten of those calls are reserved for quotes, so dashboard lookups can never starve the alert engine. Pages poll `/api/quotes` once a second. The response carries prices, day range, a sparkline built from streamed trades, the market session and the newest alert event, so a rule fired by the poller appears on screen within a second. I chose short polling over server-sent events because each open stream would pin one of waitress's worker threads.

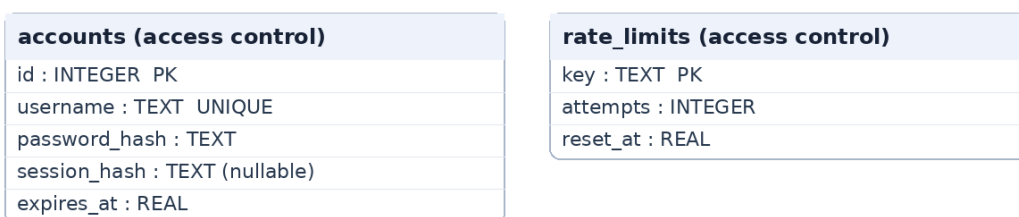
Security boundary. The API key lives only in the server environment. REST calls send it in a header, a logging filter redacts it and the browser never contacts Finnhub. Passwords are hashed with `scrypt`, sessions are revocable and stored as SHA-256 digests, and every POST requires a CSRF token. A nonce-based Content Security Policy limits scripts and network requests to this origin. Login and write rate limits are stored in SQLite so they survive restarts.

4. Data model



Logical link: same owner + ticker. No cross-domain foreign key.

Event to rule: 0 or 1 rule. Rule to events: 0 or many in the schema; the application fires each rule once. Deletion sets `rule_id` to NULL.



`UNIQUE(owner_id, ticker)` in `watchlist_items`. Account ownership is enforced by scoped queries, without foreign keys to accounts.

PK = primary key FK = foreign key Supporting tables have no domain joins.

Figure 2. The five tables in `DATA_DIR/app.db`. The dashed link between the domains is logical only.

The schema decision that matters most (ADR-3) is how the two domains relate. `alert_rules.ticker` is plain text with no foreign key to `watchlist_items`, and no query joins across the domains. Each domain's tables can move to a separate database without a schema change. The trade-off is that a rule can outlive its asset: removing a ticker makes its rules **dormant**, and adding it back reactivates them.

`alert_events` is a snapshot table. When a rule fires, the event copies the ticker, condition, threshold, observed price and UTC timestamp, and `rule_id` is set to NULL by `ON DELETE SET NULL` if the rule is

later deleted. History stays complete and readable without a join, at the cost of some denormalisation.

Firing is atomic and idempotent. `fire_rule` opens a `BEGIN IMMEDIATE` transaction, which takes SQLite's write lock up front. It deactivates the rule only if it is still active and its random `rule_token` still matches, then inserts the event in the same transaction. Two concurrent evaluations therefore produce exactly one event. A stale evaluation can't fire a new rule that reused a deleted rule's ID. A failed insert rolls the deactivation back.

Every business row carries an `owner_id`, and every query filters on the authenticated account. Tables are created at startup with `CREATE TABLE IF NOT EXISTS`, and an earlier single-user database is upgraded in a transaction, with its rows parked under owner 0, which nobody can sign in as. SQLite keeps its default journal mode.

5. Testing strategy and results

The tests concentrate on the core business logic: `is_triggered`, `AlertService.evaluate_all`, `fire_rule` and watchlist validation. Test doubles for `PriceSource` and `WatchlistReader` keep the suite deterministic and offline. Beyond the unit tests there are concurrency tests for double firing and ID reuse, access-control tests, a full end-to-end flow through the Flask test client and a contract test that launches the real python `app.py` and polls `/health`.

Coverage is measured with `pytest --cov=app --cov-report=term-missing`. Route files and `create_app` are excluded as framework glue, and `fail_under = 70` makes the build fail below the threshold. On 4 October, **240 tests passed with 100% statement coverage over 868 statements** in about 12 seconds. The known gap is that test doubles can't detect a change in Finnhub's response format, so I verified the live API by hand.

6. Compliance with the brief

Requirement	How it is met
Single process, single container	waitress plus daemon threads in one process; no broker or job runner
One start command, binds 0.0.0.0, port from env	<code>python app.py</code> ; <code>HOST</code> defaults to 0.0.0.0, <code>PORT</code> to 8080
No interactive setup, ready in seconds	Schema created and upgraded at startup; contract test checks under 5 s
SQLite at one documented path	<code>\$DATA_DIR/app.db</code> , default <code>./data/app.db</code>
One dependency manifest	<code>requirements.txt</code> with 7 packages; no Dockerfile, CI or IaC
Configurable by environment only	Every setting has a default in code; an optional <code>.env</code> is never required
External dependency is a needed public API	Finnhub only; without a key the app starts and shows prices as unavailable
Two domains, logically separable	<code>app/watchlist</code> and <code>app/alerts</code> , seam in <code>app/ports.py</code> , enforced by test
At least 70% coverage on core logic	100% of 868 statements, threshold enforced in <code>.coveragerc</code>

To run it: `uv venv`, activate it, `uv pip install -r requirements.txt`, export `PRICE_API_KEY`, then `python app.py` and open `http://localhost:8080`. The README lists every setting. For Assignment 2 the container needs a persistent volume for `DATA_DIR`, a single replica, HTTPS with `SESSION_COOKIE_SECURE=true` and a fixed `SECRET_KEY`.

7. What I chose not to build

I left out email and push notifications, password recovery and trading (ADR-5). Notifications would add an external delivery service as a runtime dependency, and trading would turn a monitoring tool into a regulated product. Fired alerts appear on screen and in the trigger history.

8. AI disclosure

I acknowledge the use of Claude Code (Claude Opus 5.5) and Codex to plan and generate most of the application code and tests and to review the project against the assignment brief. The prompts used include building the watchlist and alerts domains behind small interfaces, adding the poller and the price cache, writing tests to reach the coverage target, adding private accounts, replacing simulated prices with live Finnhub data and turning the watchlist into a real-time dashboard. The output of these prompts was used to create and revise the code in `app/`, the tests, the README, the ADR entries, the diagrams and this report after I reviewed it. Each interaction is logged in `AI_USAGE.md` with my own explanation of how the accepted code works.